Исследование методов поиска в пространстве состояний

Цель работы: приобретение навыков по работе с методами поиска в пространстве состояний с помощью языка программирования Python версии 3.x

```
PS C:\!lab1\lab1> python romania_path.py
Поиск оптимального пути от Арада до Бухареста
==================================================

Выполняем поиск A*...

Найден оптимальный путь!
Количество городов: 5
Общая дистанция: 418 км
Общая стоимость: 418

Маршрут:
1. Arad -> Sibiu (140 км)
2. Sibiu -> Rimnicu Vilcea (80 км)
3. Rimnicu Vilcea -> Pitesti (97 км)
4. Pitesti -> Bucharest (101 км)
5. Bucharest (цель достигнута!)

Путь: Arad -> Sibiu -> Rimnicu Vilcea -> Pitesti -> Bucharest

✓ Найден ожидаемый оптимальный путь!

==================================================
Информация о графе:
Всего городов: 20
Всего дорог: 23 (каждая дорога учтена дважды)
PS C:\!lab1\lab1>
```

Рисунок 1 - Результат работы программы по поиску оптимального пути от Арада до Бухареста

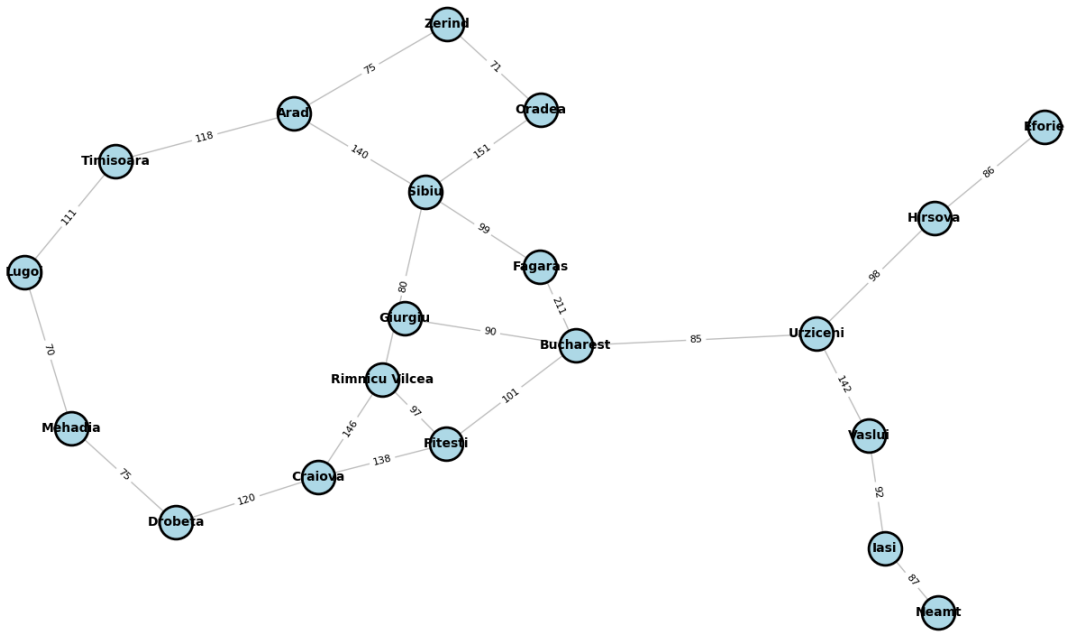

Рисунок 2 - Граф населённых пунктов Румынии

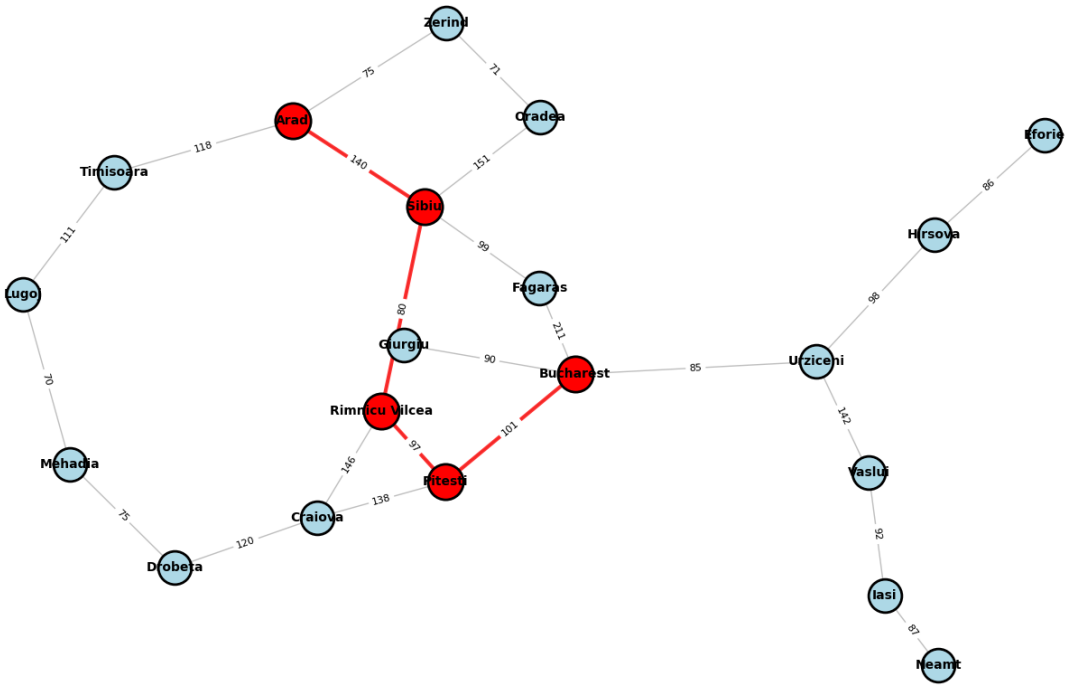

Рисунок 3 - Оптимальный путь от Арада до Бухареста

```
PS C:\!lab1\lab1> python romania_path.py
- Начинаем и заканчиваем в одном городе
- Каждый город посещается ровно один раз
- Для N городов существует (N-1)! возможных маршрутов
- Для 5 городов: 4! = 24 маршрута (быстро)
- Для 10 городов: 9! = 362,880 маршрутов (медленно)
- Для 20 городов: 19! ≈ 1.2x10¹⁷ маршрутов (невозможно)

 ОЦЕНКА ВРЕМЕНИ ВЫПОЛНЕНИЯ:
  5 городов: 24 маршрутов → ~0.00 сек
  6 городов: 120 маршрутов → ~0.00 сек
  7 городов: 720 маршрутов → ~0.01 сек
  8 городов: 5,040 маршрутов → ~0.05 сек
  9 городов: 40,320 маршрутов → ~0.40 сек
  10 городов: 362,880 маршрутов → ~3.63 сек

==================================================
Введите максимальное количество городов для анализа (5-7 рекомендовано):
```

Рисунок 4 - Программа задачи коммивояжёра

```
========================================
Анализ для 6 городов:
========================================
Города: Arad, Zerind, Oradea, Sibiu, Timisoara, Lugoj
Количество маршрутов для перебора: 120

Результаты для 6 городов:
Время выполнения: 0.000207 секунд
Проверено маршрутов: 120
Оптимальный маршрут: Arad → Zerind → Oradea → Sibiu → Timisoara → Lugoj → Arad
Общая длина: 895 км
```

Рисунок 5 - Анализ задачей коммивояжёра для 6 городов

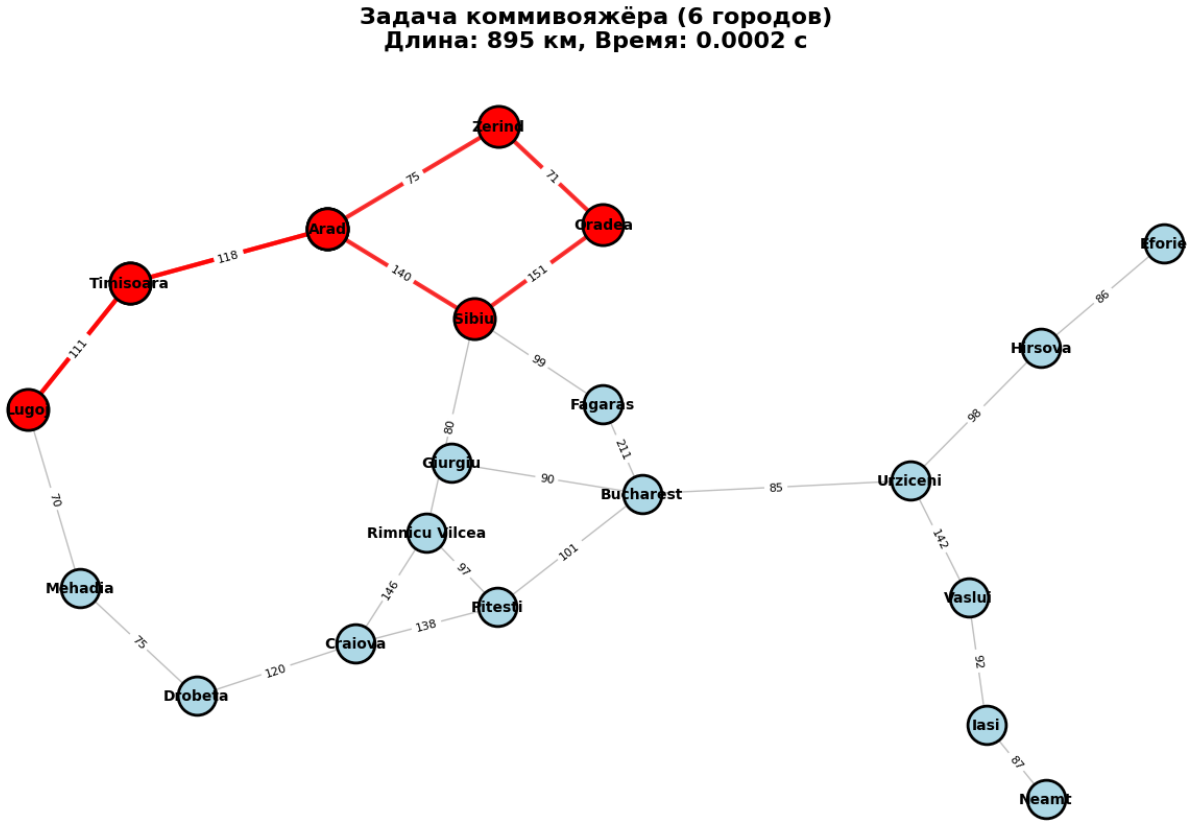

Рисунок 6 - Граф задачи коммивояжёра для 6 городов

Ответы на контрольные вопросы:

1. Перебор всех вариантов без информации о цели.
2. Эвристический использует оценку близости к цели.
3. Направляет поиск, сокращает просматриваемые узлы.
4. GPS-навигация (A* алгоритм).
5. Комбинаторный взрыв, слишком много вариантов.
6. Время, память, неполнота информации, неопределённость.
7. Найти выигрышный ход за ограниченное время.
8. Ограничивают глубину поиска, используют эвристики.
9. Начало, цель, действия, стоимость, эвристика.
10. Сравнением длины найденного пути с оптимальным.
11. Город отправления (например, Арад).
12. Узлы без потомков (нерасширенные или целевые).
13. Генерация всех возможных следующих состояний.

14. Zerind (75 км), Sibiu (140 км), Timisoara (118 км).
15. Проверка равенства состояния заданному целевому.
16. Выбор узла, проверка цели, расширение, повтор.
17. Состояние это описание ситуации, узел это объект дерева с родителем и стоимостью.
18. Возвращает возможные следующие состояния из текущего.
19. b - ширина дерева, d - глубина цели, m - ограничение по глубине.
20. Полнота означает найдётся ли решение, временной критерий означает узлы созданы, пространственный означает узлы хранятся, оптимальность это наилучшее решение.
21. Абстрактный класс для формализации задачи.
22. actions и result.
23. Проверяет, является ли состояние целевым.
24. Возвращает стоимость перехода между состояниями.
25. Хранит информацию о состоянии, родителе, действии и стоимости пути.
26. state, parent, action, path_cost.
27. Неудачу поиска.
28. Генерирует дочерние узлы для текущего узла.
29. Последовательность действий от корня к текущему узлу.
30. path_states возвращает состояния, path_actions действия.
31. deque из collections.
32. FIFO очередь (первым пришел — первым ушел), LIFO стек (последний пришел — первый ушел).
33. Добавляет элемент с учётом приоритета (использует heapq).
34. Когда нужно обрабатывать элементы в порядке приоритета (например, A*).
35. Извлекает элемент с наивысшим приоритетом из кучи.

Вывод: в процессе выполнения лабораторной работы мы приобрели навыки по работе с методами поиска в пространстве состояний с помощью языка программирования Python версии 3.x